SUPERVISED MACHINE LEARNING

Polynomial Regression

Jean René RANDRIANJAKAMANANA

2024-11-25

Table de matières

B) POLYNOMIAL REGRESSION y=ax²+bx+c

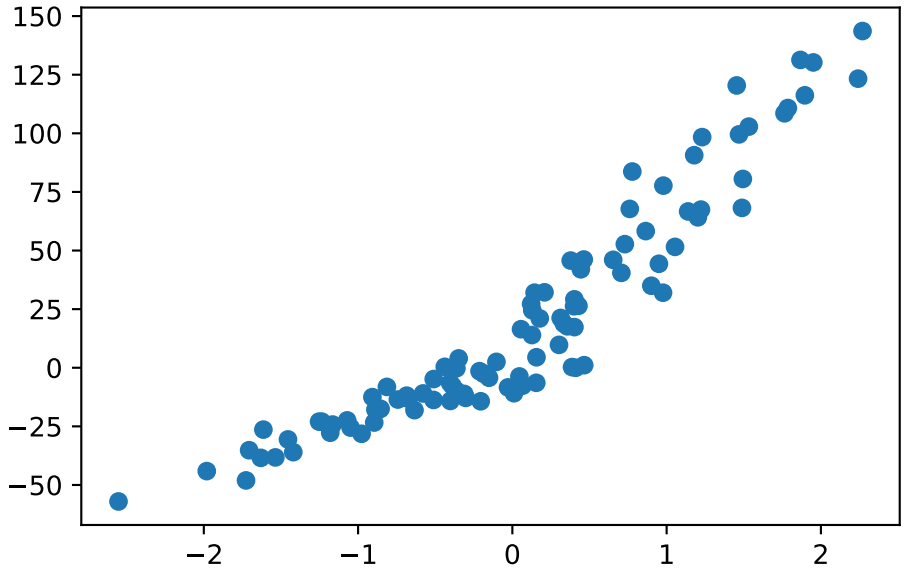
1) Model ameliorated through Gradient Descent

Ici, la matrice des variables explicatives X est formée par: X = $\left[x^2, x, un\right]$ Aussi β va être de trois dimensions composée par $\beta = \begin{pmatrix} a \\ b \\ c \end{pmatrix}$

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.datasets import make_regression
# from sklearn.linear_model import LinearRegression
# from sklearn.metrics import mean_squared_error, r2_score
```

a) Importation of datasets and plot

```

np.random.seed(0)
x, y = make_regression(n_samples = 100, n_features = 1, noise = 10)
y = y + abs(y/2)

plt.close()
plt.scatter(x,y)
plt.show()
```



b) Dimension and resize of data

```
# Y = X*beta + epsilon
#Y_ajust = X*beta_ajustr
y.shape
x.shape
y = y.reshape(y.shape[0],1)
```

c) Random estimation of the model's parameters

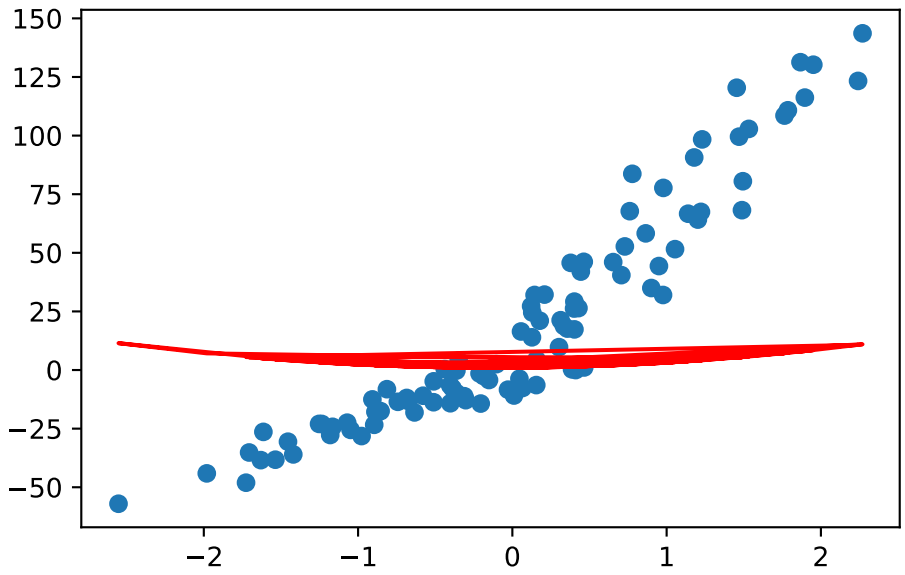
```
np.random.seed(0)
X = np.hstack((x, np.ones(x.shape)))

# Matrix X with constant
X = np.hstack((x**2, X))

# Polynomial degree 2
beta = np.random.randn(3,1)
beta
```

```
array([[1.76405235],
       [0.40015721],
       [0.97873798]])
```

d) Model fonction cretion and Regression plot

```
def modele(X,beta):
  return X.dot(beta)

plt.close()
```

```
plt.scatter(x,y)
plt.plot(x, modele(X, beta), c ='r')
plt.show()
```



e) Loss function creation

```
def cost_function(X, y, beta):
  m = len(y)
  return (1/2*m)*np.sum((modele(X,beta) - y)**2)

cost_function(X,y,beta)
```

```
11815228.81241695
```

f) Minimisation of loss function: Gradient Descent

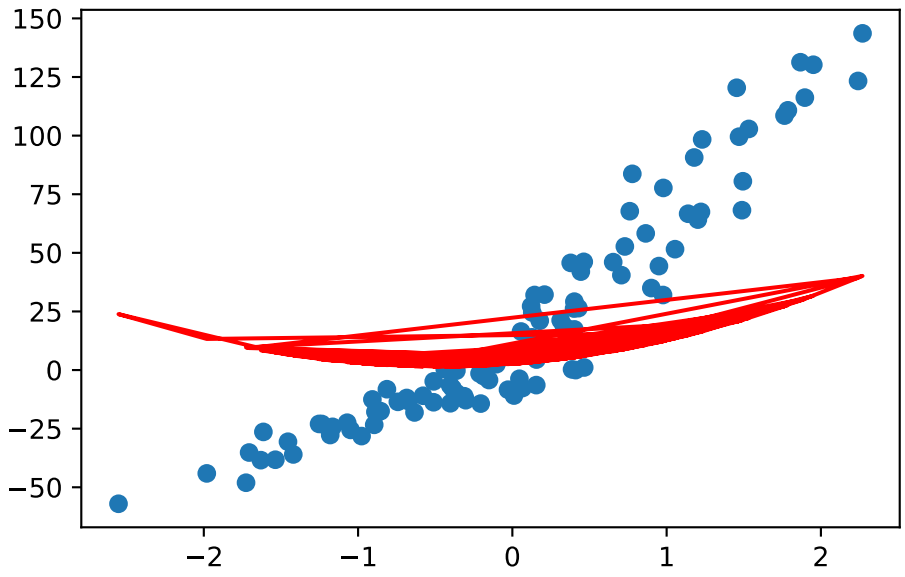
```
#Gradient functio
def grad_cost_funct(X, y, beta):
  m = len(y)
  return (1/m)*X.T.dot(modele(X,beta) - y)

#Gradient descent algorithm
def gradient_descent(X,y,beta, learning_rate, n_iterations):
  cost_history = np.zeros(n_iterations)
  for i in range(0, n_iterations):
    beta = beta - learning_rate*grad_cost_funct(X,y,beta)
    cost_history[i] = cost_function(X,y,beta)
    return beta, cost_history
```

g) Model training

```
beta_final, cost_history = gradient_descent(X,y,beta, learning_rate = 0.1, n_iterations = 1000)
beta_final
```

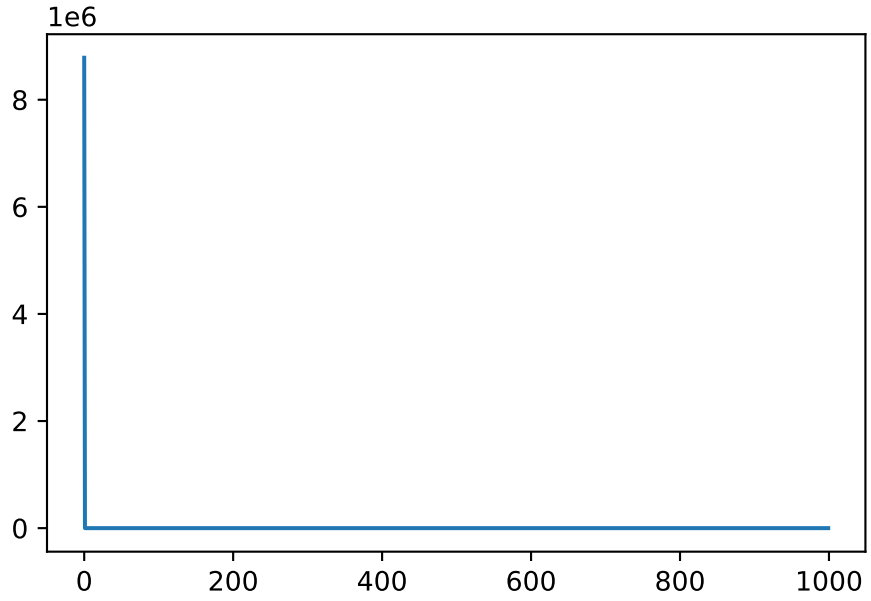
```
array([[5.14840613],
       [4.83613059],
       [2.64753046]])
```

```
y_predi = modele(X, beta_final)
plt.close()
plt.scatter(x, y)
plt.plot(x, y_predi, c='r')
plt.show()
```



h) Learning curve (Decreasing plot of the loss function)

```
plt.close()
plt.plot(range(1000), cost_history)
plt.show()
```



i) Coefficient of determination (R2)

```
def coeff_determination(y, y_predi):
  u = ((y-y_predi)**2).sum()
  v = ((y-np.mean(y))**2).sum()
  return (1-u/v)

R2 = coeff_determination(y, y_predi)
print(f"R2 = {R2} or {round(R2,5)*100} %")
```

```
R2 = 0.19567277848245435 or 19.567 %
```

2) Model with sklearn

a) With PolynomialFeatures and LinearRegression

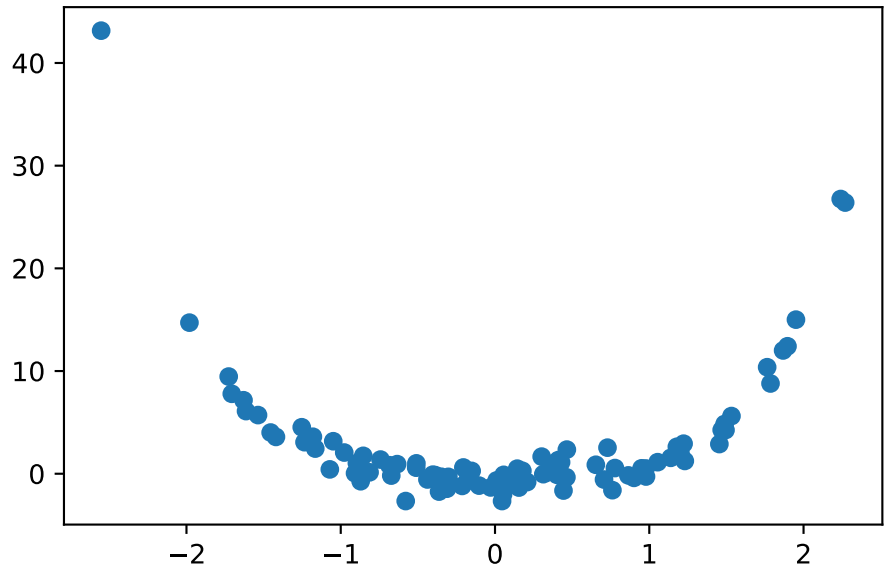
Here, we use the polynomial degree 4

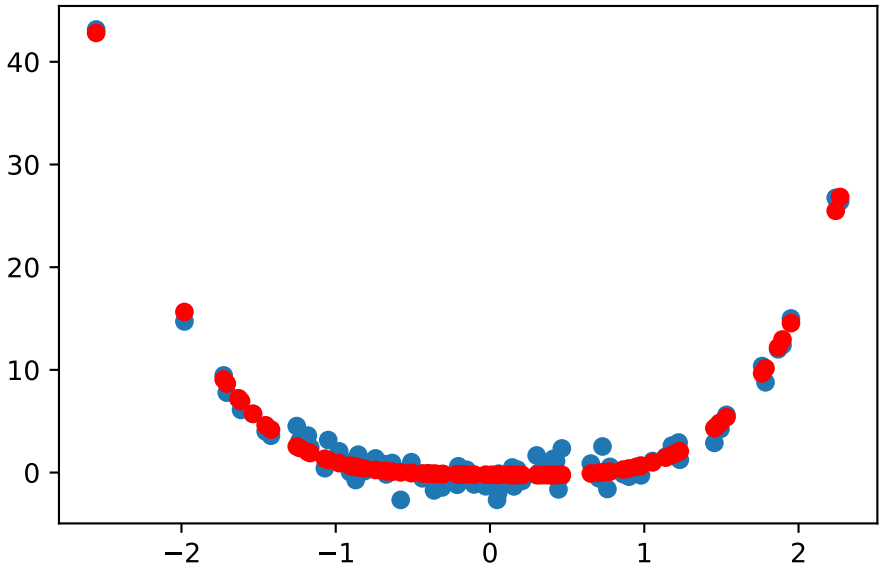
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.datasets import make_regression
from sklearn.metrics import mean_squared_error, r2_score
```

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
```

```
np.random.seed(0)
X, y = make_regression(n_samples = 100, n_features = 1, noise = 10)
y = X**4 + np.random.randn(100,1)
plt.close()
plt.scatter(X,y)
```

```
plt.show()

# Features of polynomial
poly = PolynomialFeatures(degree = 4, include_bias = False)

# Creating the new features of the matrix X
# fit to 2D array
# transform to [X, X^2]
poly_features = poly.fit_transform(X.reshape(-1,1))

# Linear Regression
reg = LinearRegression()
reg.fit(poly_features, y)

reg.coef_
reg.intercept_
y_pred = reg.predict(poly_features)

R2 = r2_score(y, y_pred)
rmse = mean_squared_error(y, y_pred)

#Plot
R2
rmse

plt.close()
plt.scatter(x, y)
plt.scatter(x, y_pred, c='r')
plt.show()
```





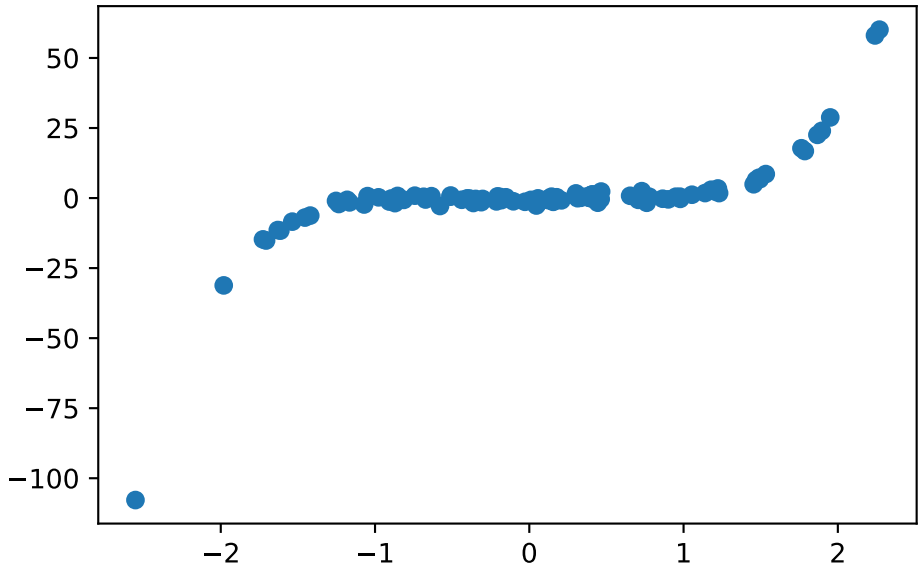
b) With SVR (Support Vector Regression) of SVM (Support Vector Machine)

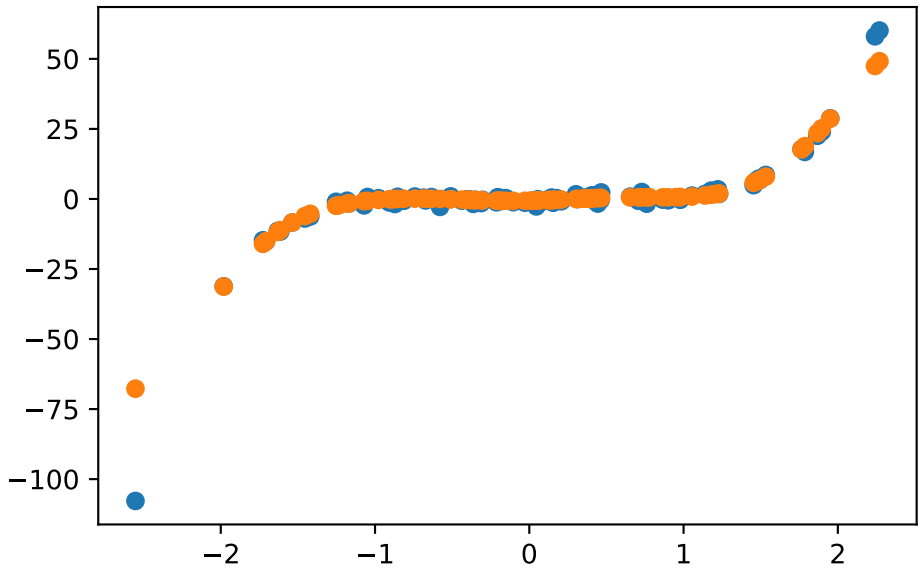
The SVR model can assure a polynomial regression model, but the high the degree of the polynomial is, the worst the quality of the model compared to the one of PolynomialFeatures and LinearRegression.

Here, we use a degree of 5.

```
from sklearn.svm import SVR
```

```
# Dataset creatio
np.random.seed(0)
X, y = make_regression(n_samples = 100, n_features = 1, noise = 10)
y = X**5 + np.random.randn(100,1) # Degree 5

plt.close()
plt.scatter(X,y)
plt.show()

# SVR
svr = SVR(C = 100)

y = y.ravel()
svr.fit(X, y)

y_pre = svr.predict(X)
R2 = r2_score(y, y_pre)
rmse = mean_squared_error(y, y_pre)
R2
rmse
plt.close()
plt.scatter(X, y)
plt.scatter(X, y_pre)
plt.show()
```


50
25
0
−25
−50
−75
−100
−2
−1
0
1
2


50
25
0
−25
−50
−75
−100
−2
−1
0
1
2